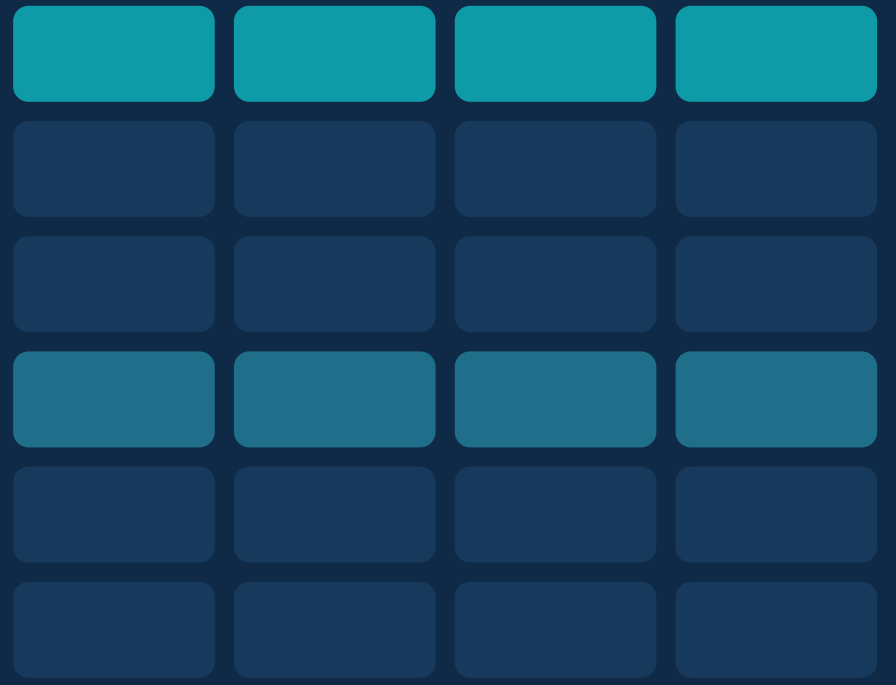
CE-2103 · LECCIÓN 08

Bases de Datos

De los archivos a SQL y NoSQL



Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

¿Qué aprenderemos hoy?

1. Qué es un DBMS y por qué reemplazó a los archivos
2. Cómo se modelan los datos en el modelo relacional
3. Cómo los índices (árboles B+) aceleran las búsquedas
4. Cómo consultar y modificar datos con SQL
5. Qué ofrecen las bases de datos NoSQL y cuándo usarlas

De un dato a un DBMS

1

Dato

Representación simbólica de un hecho o atributo. Ej.: la edad de una persona o el nombre de un producto.

2

Base de datos

Colección de datos organizada para un propósito específico, dentro de un dominio concreto.

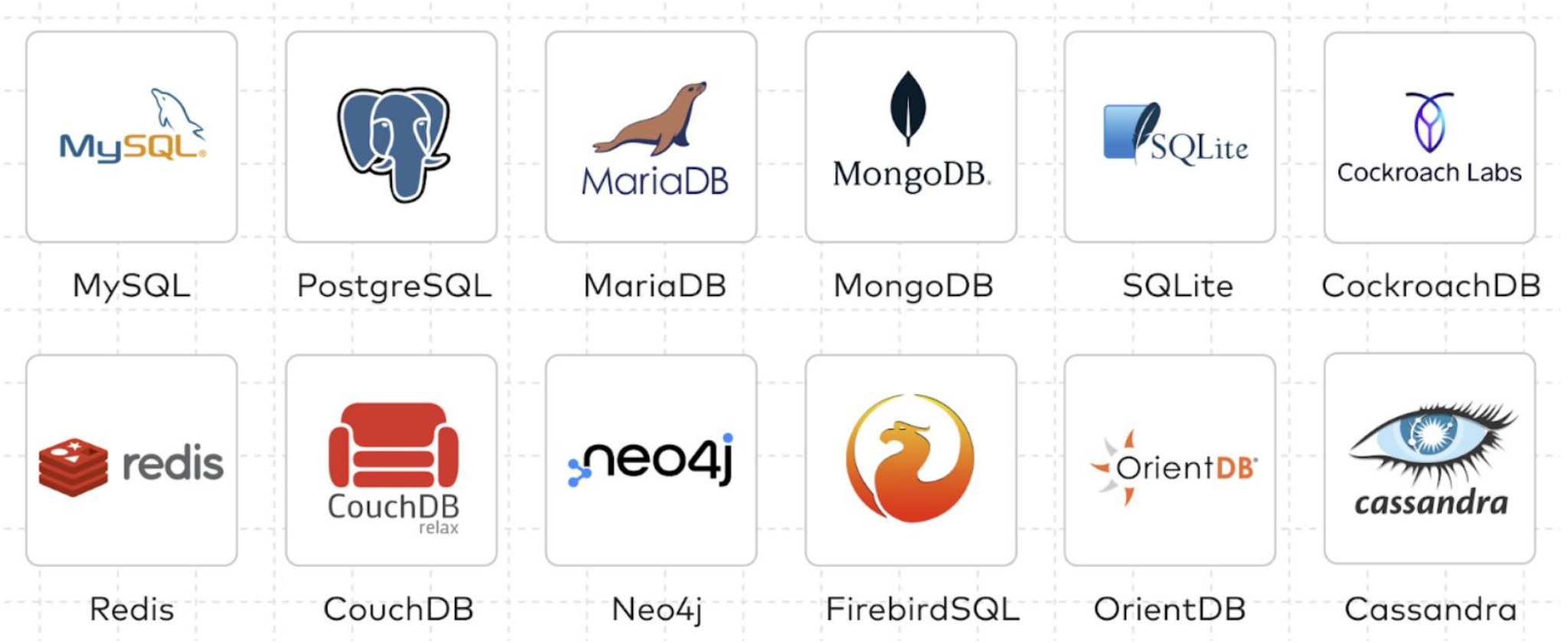
3

DBMS

Software que permite crear, leer, actualizar y eliminar datos de una base de datos.

Recuerde: no hay bases de datos genéricas. Cada una describe un dominio (universo de discurso) específico.

Algunos DBMS populares



Cada DBMS adopta un enfoque: relacional, orientado a objetos o NoSQL.

Antes de los DBMS: archivos

Cada programa leía y escribía sus propios archivos. Al crecer el sistema aparecen problemas:

Formatos inconsistentes

Cada archivo define su propio formato.

Redundancia

Los mismos datos duplicados en archivos separados.

Diseño ineficiente

Estructuras pensadas para un solo programa.

Inseguridad

Poco control sobre quién accede a los datos.

Acoplamiento alto

Cambiar un archivo obliga a cambiar los programas.

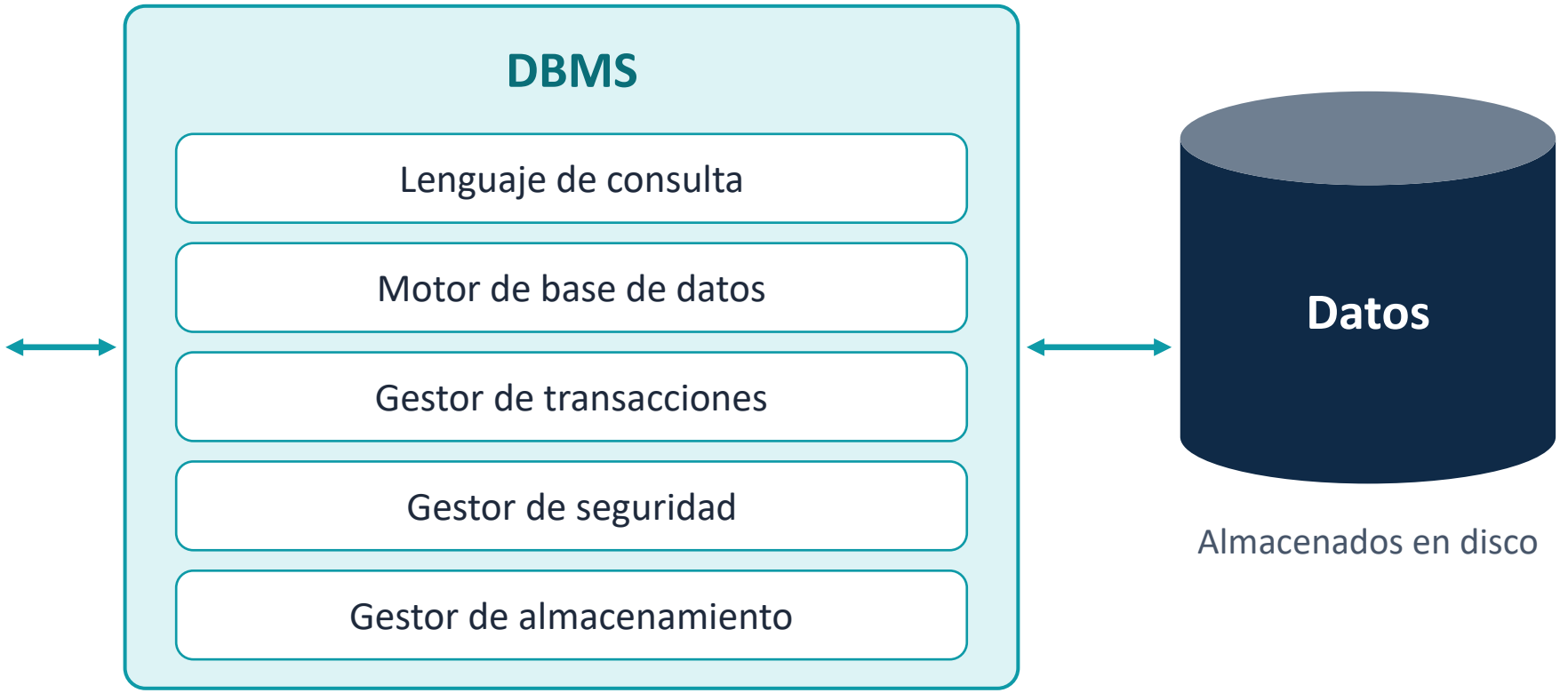
Integridad frágil

Un error en un programa puede corromper los datos.

Enfoque de bases de datos: una capa de indirección

Programas y usuarios

Envían consultas y comandos



Almacenados en disco

Independencia de los datos: los programas no necesitan conocer la estructura interna. El DBMS es auto-descriptivo (guarda metadatos).

Ventajas del enfoque de bases de datos

Independencia de datos

Los programas no dependen de la estructura interna de los datos.

Integridad

Reglas que garantizan datos precisos y consistentes.

Seguridad

Control de acceso y protección contra accesos no autorizados.

Concurrencia

Acceso simultáneo de múltiples usuarios sin conflictos.

Escalabilidad

Maneja grandes volúmenes de datos y crece con la organización.

Datos compartidos

Varias aplicaciones usan los mismos datos sin duplicarlos.

Cada base de datos adopta un enfoque

Como los lenguajes de programación con sus paradigmas, las bases de datos siguen un enfoque:

1

Relacional

Tablas de filas y columnas relacionadas por claves. Modelo de Edgar Codd (1970).

2

Orientada a objetos

Los datos se modelan como objetos con atributos y métodos.

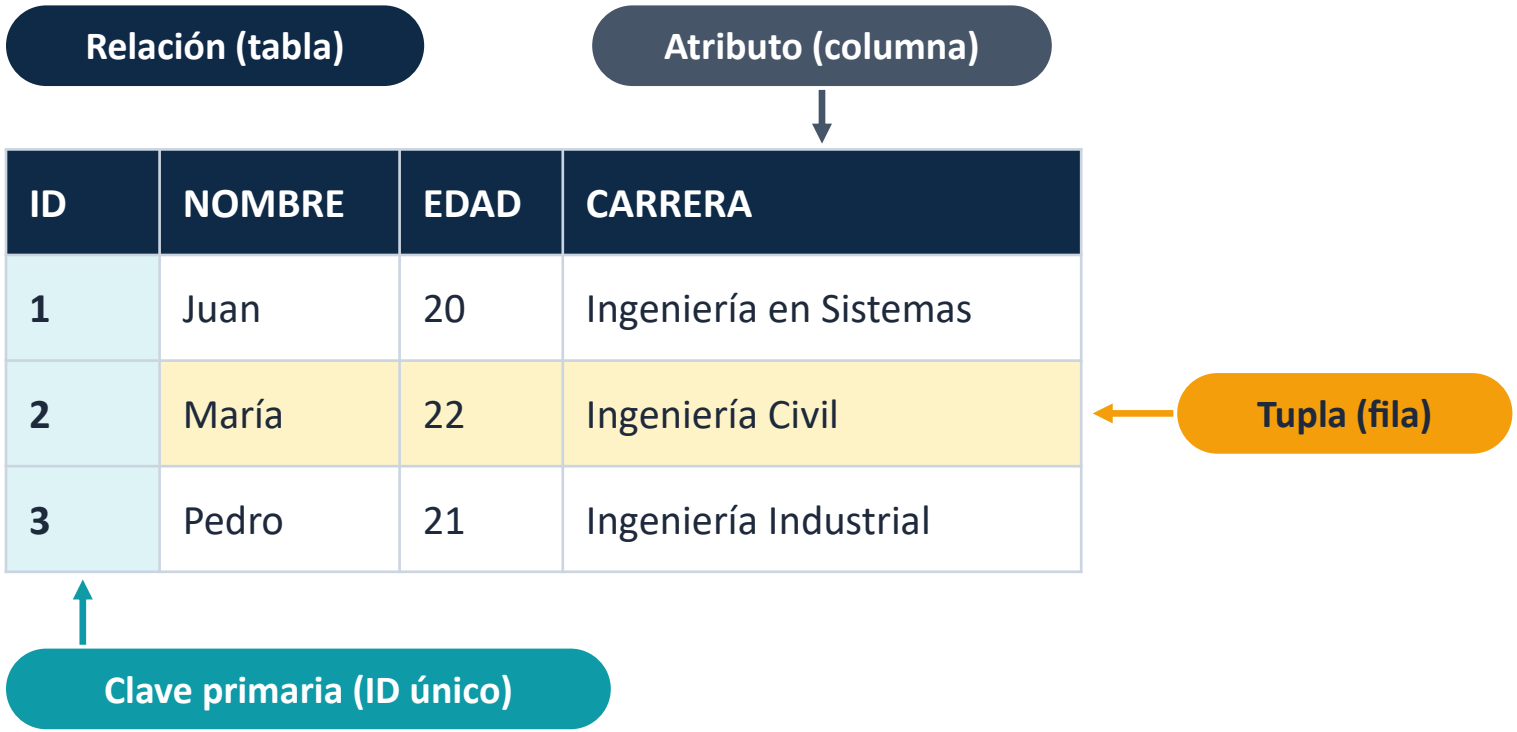
3

NoSQL

Alternativa flexible y escalable para datos no estructurados o semiestructurados.

> Un DBMS puede soportar más de un enfoque (p. ej., Oracle: relacional y objetos). Por eso decimos que la **base de datos** sigue un enfoque, no el DBMS.

Anatomía de una tabla



Otros términos

Esquema: estructura de la tabla (como una clase en POO).

Clave foránea: referencia a la clave primaria de otra tabla.

Clave candidata: atributo que podría ser clave primaria.

Relaciones muchos-a-muchos

ESTUDIANTE

ID	NOMBRE
1	Juan
2	María
3	Pedro

FK

MATRICULA (tabla intermedia)

ESTUDIANTE_ID	CURSO_ID
1	1
1	2
2	1

FK

CURSO

ID	NOMBRE
1	Matemáticas
2	Física
3	Química

PK clave primaria **FK clave foránea**

Un estudiante matricula **muchos** cursos y un curso tiene **muchos** estudiantes. La **tabla intermedia** guarda solo las claves (sin duplicar datos del estudiante ni del curso).

Integridad referencial y normalización

Integridad referencial

Restricción que valida que cada clave foránea apunte a una clave primaria existente.

Ej.: `MATRICULA.ESTUDIANTE_ID` debe existir en `ESTUDIANTE.ID`.

vs

Normalización

Proceso de diseño del esquema, basado en las formas normales, que reduce la redundancia de datos.

Se aplica al diseñar las tablas.

Ojo: la normalización **no** es consecuencia de la integridad referencial. Son conceptos distintos que se complementan.

Transacciones ACID

Una transacción es una operación que modifica los datos y debe cumplir cuatro propiedades:

A

Atomicidad

Todo o nada: si una parte falla, se deshace la transacción completa.

C

Consistencia

Lleva la base de un estado consistente a otro, respetando las restricciones.

I

Aislamiento

Las transacciones concurrentes no interfieren entre sí.

D

Durabilidad

Los cambios confirmados persisten aun después de un fallo del sistema.

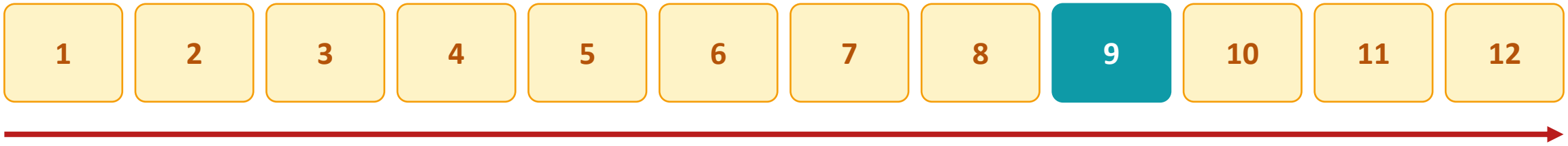
El problema: full table scan

```sql
SELECT *
FROM ESTUDIANTE
WHERE NOMBRE = 'María';
```

O(n)

costo del recorrido completo

Tabla ESTUDIANTE con n filas



El DBMS revisa **todas** las filas, una por una, comparando `NOMBRE` con 'María'.

> Sin índice, el DBMS no sabe dónde están las filas que cumplen la condición. Con millones de registros resulta prohibitivamente lento.

¿Qué es un índice?

Estructura auxiliar que asocia el valor de una columna (la **clave**) con la ubicación de sus filas.

Índice (por NOMBRE)

NOMBRE	Apunta a
Juan	fila 1
María	fila 2
Pedro	fila 3

Tabla ESTUDIANTE

ID	NOMBRE	EDAD
1	Juan	20
2	María	22
3	Pedro	21

> **Analogía:** el índice de un libro. En vez de leer todas las páginas, busca el término y va directo a la página.

Árbol B+

Nodos con muchas claves: árbol poco profundo y pocos accesos a disco. Sus hojas enlazadas permiten recorrer rangos.

Costo de búsqueda

$O(\log n)$ en vez de $O(n)$.

Árbol B+ vs. índice hash

Árbol B+

- Igualdad en `O(log n)`
- Soporta rangos: `WHERE EDAD > 20`
- Permite recorridos ordenados
- Predeterminado en la mayoría de DBMS

vs

Hash

- Igualdad exacta en `O(1)` promedio
- No soporta rangos
- No preserva el orden de las claves
- Útil solo para búsquedas exactas

El árbol B+ es la misma familia de estructuras que estudiamos en almacenamiento externo.

Clustered vs. non-clustered

	Clustered (agrupado)	Non-clustered (secundario)
Almacenamiento	La tabla se ordena físicamente según la clave; las filas son las hojas del árbol.	Estructura separada con las claves ordenadas y un puntero a la fila real.
Acceso al dato	Directo, sin salto adicional.	Un salto extra hasta la fila.
Cantidad por tabla	Uno solo.	Varios.

> Al declarar una **clave primaria**, el DBMS suele crear un índice sobre ella (frecuentemente clustered).

No conviene indexar todo

Lecturas más rápidas

Evitan el full table scan en consultas con `WHERE`, `JOIN` u `ORDER BY`.

Escrituras más lentas

Cada `INSERT`, `UPDATE` o `DELETE` también actualiza los índices afectados.

Más espacio en disco

Cada índice es una estructura aparte que ocupa almacenamiento.

Regla práctica: indexe columnas de uso frecuente en `WHERE`, `JOIN` u `ORDER BY`. Evítelos en tablas con muchas escrituras o columnas que casi no se consultan.

Crear un índice en SQL

Crear el índice

```
CREATE INDEX idx_estudiante_nombre
ON ESTUDIANTE (NOMBRE);
```

La misma consulta de antes

```
SELECT *
FROM ESTUDIANTE
WHERE NOMBRE = 'María';
```

Sin índice

Full table scan: revisa las n filas.

O(n)

Con índice

Búsqueda en el árbol B+ del índice.

O(log n)

La diferencia es despreciable en tablas pequeñas, pero determinante cuando n es grande.

SQL: DDL y DML

DDL · Data Definition Language

Define y modifica las **estructuras** de la base de datos.

CREATE ALTER DROP

DML · Data Manipulation Language

Manipula los **datos** dentro de esas estructuras.

SELECT INSERT UPDATE DELETE

> SQL (Structured Query Language) es el lenguaje estandarizado para interactuar con bases de datos relacionales.

DDL: CREATE TABLE

ESTUDIANTE

```sql
CREATE TABLE ESTUDIANTE (
  ID INT PRIMARY KEY,
  NOMBRE VARCHAR(50),
  EDAD INT,
  CARRERA VARCHAR(50)
);
```

CURSO

```sql
CREATE TABLE CURSO (
  ID INT PRIMARY KEY,
  NOMBRE VARCHAR(50)
);
```

MATRICULA (tabla intermedia)

```sql
CREATE TABLE MATRICULA (
  ESTUDIANTE_ID INT,
  CURSO_ID INT,
  PRIMARY KEY (ESTUDIANTE_ID, CURSO_ID),
  FOREIGN KEY (ESTUDIANTE_ID)
    REFERENCES ESTUDIANTE(ID),
  FOREIGN KEY (CURSO_ID)
    REFERENCES CURSO(ID)
);
```

> Los tipos de datos varían según el DBMS. Las restricciones también pueden agregarse después con `ALTER TABLE`.

DML: INSERT, UPDATE y DELETE

INSERT · agregar filas

```
INSERT INTO CURSO (ID, NOMBRE)
VALUES (1, 'Matemáticas');
```

UPDATE · modificar filas

```
UPDATE ESTUDIANTE
SET EDAD = 21
WHERE ID = 1;
```

DELETE · eliminar filas

```
DELETE FROM CURSO
WHERE ID = 1;
```

Cuidado: sin `WHERE`, `UPDATE` y `DELETE` afectan **todas** las filas de la tabla.

Integridad: al borrar en una tabla «padre», los registros «hijos» se eliminan o actualizan (acción en cascada).

DML: SELECT

Estructura

```
SELECT columna1, columna2
FROM tabla
WHERE condición;
```

Todos los estudiantes

```
SELECT *
FROM ESTUDIANTE;
```

Cursos de un estudiante específico

```
SELECT C.*
FROM CURSO C
JOIN MATRICULA M ON C.ID = M.CURSO_ID
WHERE M.ESTUDIANTE_ID = 1;
```

> `*` significa «todas las columnas». `C.*` son todas las columnas de `CURSO` (alias `C`).

JOIN: combinar tablas

Consulta

```
SELECT E.NOMBRE, C.NOMBRE
FROM ESTUDIANTE E
JOIN MATRICULA M
    ON E.ID = M.ESTUDIANTE_ID
JOIN CURSO C
    ON C.ID = M.CURSO_ID;
```



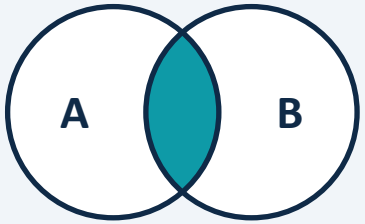
Resultado

E.NOMBRE	C.NOMBRE
Juan	Matemáticas
Juan	Física
María	Matemáticas

La tabla intermedia `MATRICULA` conecta cada estudiante con sus cursos. Cada fila de `MATRICULA` produce una fila en el resultado.

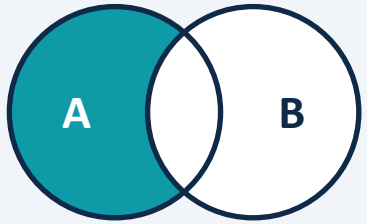
Tipos de JOIN

Combinan filas de dos tablas (A y B) según una condición `ON`. El área coloreada es lo que se devuelve.



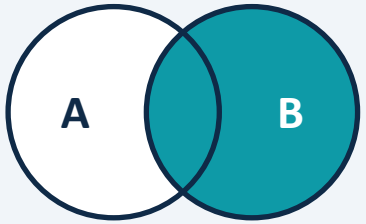
INNER JOIN

Solo las filas con coincidencia en ambas tablas.



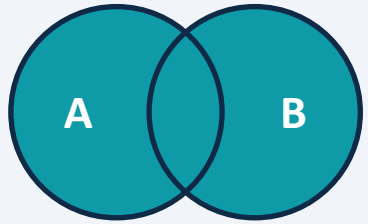
LEFT JOIN

Todas las filas de A, más las coincidencias de B.



RIGHT JOIN

Todas las filas de B, más las coincidencias de A.



FULL OUTER JOIN

Todas las filas de A y de B, coincidan o no.

> En la práctica, `JOIN` a secas equivale a `INNER JOIN`.

NoSQL: origen y características

1998

Primer uso del nombre

Carlo Strozzi llamó «NoSQL» a su base de datos ligera, que literalmente no usaba SQL.

2009

Nuevo significado

Se reinterpreta como «Not Only SQL»: existen alternativas al modelo relacional.

Estructura flexible

No requiere un esquema fijo.

Escalabilidad horizontal

Reparte la carga entre varios servidores o nodos.

Modelos diversos

Documentos, grafos, columnas y clave-valor.

Tipos de bases de datos NoSQL

Documentos

Documentos JSON o BSON agrupados en colecciones (similares a tablas).

MongoDB · CouchDB

Grafos

Nodos y relaciones entre ellos. Ideales para relaciones complejas.

Neo4j · Neptune

Columnas

Datos por columnas en vez de filas. Eficientes en análisis y agregaciones.

Cassandra · HBase

Clave-valor

Pares clave-valor con claves únicas. Lecturas y escrituras muy rápidas.

Redis · DynamoDB

> Cada tipo está optimizado para un tipo distinto de aplicación y carga de trabajo.

Una misma información, dos modelos

Relacional

- Datos repartidos en varias tablas
- Se reconstruyen con `JOIN`
- Esquema fijo y definido de antemano

```
SELECT E.NOMBRE, C.NOMBRE
FROM ESTUDIANTE E
JOIN MATRICULA M ON E.ID = M.ESTUDIANTE_ID
JOIN CURSO C ON C.ID = M.CURSO_ID;
```

vs

Documentos

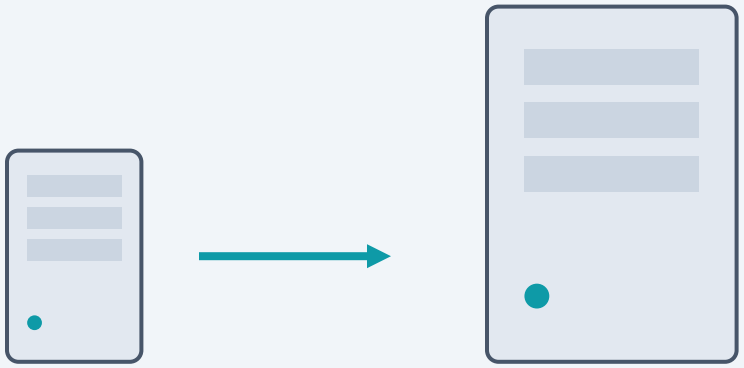
Todo en un solo documento; su estructura puede variar entre documentos.

```
{
  "id": 1,
  "nombre": "Juan",
  "edad": 20,
  "carrera": "Ingeniería en Sistemas",
  "cursos": ["Matemáticas", "Física"]
}
```

Escalar hacia arriba o hacia afuera

Escalar hacia arriba (scale up)

Se mejora un solo servidor: más CPU, memoria y disco.



Más simple, pero con límite de hardware

Escalar hacia afuera (scale out)

Se agregan más servidores que trabajan en conjunto.



NoSQL: crece agregando nodos

Las bases NoSQL están pensadas para **escalar horizontalmente**: basta con agregar más servidores.

Relacional vs. NoSQL

	Relacional	NoSQL
Esquema	Rígido, definido de antemano	Flexible
Modelo	Tablas, filas y columnas	Documentos, grafos, columnas, clave-valor
Consistencia	Transacciones ACID	Algunas sacrifican consistencia (eventual)
Escalabilidad	Normalmente vertical	Horizontal, agregando nodos
Ecosistema	Muy maduro	En crecimiento, menos maduro

Casos de uso NoSQL: aplicaciones web escalables y análisis de datos en tiempo real (p. ej., Elasticsearch).

Resumen

- Un **DBMS** gestiona los datos y aporta independencia, integridad, seguridad y concurrencia.
- El **modelo relacional** organiza los datos en tablas unidas por claves primarias y foráneas.
- Los **índices** (árboles B+) aceleran las lecturas de `O(n)` a `O(log n)`, a costa de escrituras y espacio.
- **SQL** define (DDL) y manipula (DML) los datos; `JOIN` combina tablas.
- **NoSQL** ofrece esquemas flexibles y escalabilidad horizontal.

Referencias: Gillenson, M. (2012). Fundamentals of Database Management Systems. John Wiley & Sons. · Sullivan, M. (2019). NoSQL for Mere Mortals. Addison-Wesley.